LLM 如何编程（底层原理）

第 02 讲 · 2026 年 9 月 3 日

layout: statement

理解了底层原理的开发者拥有*巨大的*优势。

layout: default

四个核心概念

今天的底层原理

<ph-text-aa-bold class="text-3xl text-blue-600 mb-3" />
分词
模型如何"看"你的代码。
<ph-resize-bold class="text-3xl text-blue-600 mb-3" />
上下文窗口
唯一重要的资源。
<ph-eye-slash-bold class="text-3xl text-blue-600 mb-3" />
幻觉
模型为什么会产生虚构内容。

layout: section

分词

模型看到的内容

layout: default

BPE 直觉

是 tokens，不是 words

- 模型看不到字符
- 模型看不到单词
- 模型看到的是 **tokens** —— 它在训练中见过的片段

字节对编码：合并最常见的相邻配对，重复约 100K 词表大小。

layout: default

字节对编码，实操演示

`getOrderById` 去了哪里

```
1. 从字符开始
   g e t O r d e r B y I d                          12 个 tokens

2. 合并最常见的相邻配对。重复，约 100K 次
   e+r -> er       g+et -> get       O+rder -> Order

3. 在训练中幸存下来的合并结果就是词表
   get  Order  By  Id                                4 个 tokens
```

没有人选择了这四个。它们只是足够常见，挣到了一个位置。

layout: default

o200k_base · GPT-4o 和 5 的编码方式

真实的 tokens，真实的 ID

const order = await getOrderById(orderId);
1671
const
2569
·order
314
·=
4021
·await
717

layout: default

两个后果

分词带来的代价

<ph-currency-dollar-bold class="text-3xl text-blue-600 mb-3" />
成本单位
冗长的代码更贵——在供应商那里是美元，在自己的机器上是 RAM 和秒数。
<ph-eye-bold class="text-3xl text-blue-600 mb-3" />
注意力单位
每个 token 都在争夺注意力。你的标识符被拆成常见片段，所以它们没有特殊权重。

layout: two-col

9 个 tokens

```
def f(x):
    return x * 2
```

::right::

约 35 个 tokens

```
def calculate_double_of_input_value(
    input_value: int
) -> int:
    """Return value times two."""
    return input_value * 2
```

layout: section

上下文窗口

唯一重要的资源

layout: default

窗口内外

模型看到的内容

- **窗口内**：文件、历史记录、代码库地图、系统提示词
- **窗口外**：*什么都没有*
- 没有数据库。没有网页抓取。没有检索

这对工程师来说是关于 LLM 最重要的事实。

layout: default

为什么更大 ≠ 更好

大上下文的三重代价

<ph-currency-dollar-bold class="text-3xl text-blue-600 mb-3" />
成本
按 token 计费，或自己机器上的 RAM 和秒数。注意力是二次方的：2 倍上下文 ≈ 4 倍工作量。
<ph-clock-bold class="text-3xl text-blue-600 mb-3" />
延迟
笔记本上第一个 token 要等几十秒，而一个简洁的提示词一开始就能输出。
<ph-warning-bold class="text-3xl text-rose-600 mb-3" />
质量
大海捞针：更大可能*更差*

layout: default

生成的一个步骤

权衡整个窗口

<svg viewBox="0 0 900 250" class="w-full mt-4" role="img" aria-label="一个 token 正在生成，带有指向窗口中已有每个 token 的加权链接"> <path d="M450 82 Q 276 145 102 176" fill="none" stroke="var(--c-primary)" stroke-width="1.9" opacity="0.22"/> <path d="M450 82 Q 334 145 218 176" fill="none" stroke="var(--c-primary)" stroke-width="5.2" opacity="0.59"/> <path d="M450 82 Q 392 145 334 176" fill="none" stroke="var(--c-primary)" stroke-width="1.5" opacity="0.17"/> <path d="M450 82 Q 450 145 450 176" fill="none" stroke="var(--c-primary)" stroke-width="2.1" opacity="0.24"/> <path d="M450 82 Q 508 145 566 176" fill="none" stroke="var(--c-primary)" stroke-width="1.6" opacity="0.19"/> <path d="M450 82 Q 566 145 682 176" fill="none" stroke="var(--c-primary)" stroke-width="5.8"

layout: default

为什么成本增长比上下文更快

每个 token 都关注其他所有 token

	窗口中的 tokens 数	评分的配对对数	相对工作量
	1K	1,000,000	1×
	2K	4,000,000	4×
	8K	64,000,000	64×
	32K	1,024,000,000	*1024*×

上下文翻倍，工作量翻四倍。窗口是预算，不是容器。

layout: default

/add 的教学意义

Aider 的摩擦就是课程

```
/add main.py          → 文件进入上下文
/drop main.py         → 文件离开上下文
/tokens               → 精确查看窗口中有什么
/clear                → 重置历史（保留文件）
```

这不是限制。这是你本来就该有的纪律。

layout: default

长会话

上下文漂移

- 历史不断积累：你发送的每一条消息，它返回的每一条消息
- 五十条消息后，窗口中大部分都是*死重*
- 模型无法区分什么是现在真实的，什么是第 3 条消息时真实的

任务之间用 `/clear` 。Claude Code（L05）会帮你压缩，而压缩是有损的。

layout: statement

聊天休息 · 5 分钟

和邻座聊聊：当一个长聊天*退化*时，窗口内部发生了什么？

layout: section

幻觉

一个误导性的词

layout: default

实际发生了什么

模型在做它被训练来做的事

<v-clicks>

- 自回归采样
- 从分布中挑选最可能的下一个 token
- 没有事实验证步骤
- 没有数据库查询
- 没有"等等，这个存在吗？"

</v-clicks>

layout: default

为什么是 "requests" 而不是 "httpx"

当上下文沉默时，统计先验获胜

<v-clicks>

- 训练数据：`import requests` 比 `import httpx` 多 100 倍
- 你的项目使用 httpx
- 提示词没有提到它
- 模型输出 `requests` —— 看似合理，但对于*你的*上下文来说是错的

</v-clicks>

layout: default

代码中的三种类型

常见的幻觉

<ph-function-bold class="text-3xl text-rose-600 mb-3" />
虚构 API
user.get_profile() —— 看似合理，不是真实的。
<ph-clock-counter-clockwise-bold class="text-3xl text-rose-600 mb-3" />
过时的知识
在 2.0 代码库中使用 pandas 1.x API。
<ph-warning-circle-bold class="text-3xl text-rose-600 mb-3" />
错误的签名
错误的 kwarg，错误的 dtype —— 很微妙。

layout: statement

当模型需要一个事实时，*给它看这个事实*。

layout: default

局限

上下文无法修复什么

- "我们内部库的最新版本是什么？"
- 答案不在你的代码库里，所以任何 `/add` 都得不到它
- 模型猜测，*看似合理*，而你看不出来

把事实交给它，或者构建一个查找事实的工具。你将在第 5 周构建这些。

layout: section

能力 vs 知识

两个维度

layout: two-col

能力

推理技能。多步骤、跨文件、保持约束。
随以下增长：模型大小。**
qwen3.5:4b → 9b → 前沿模型。
::right::

知识

模型阅读了什么。时效性。小众深度。
随以下增长：训练数据大小 + 新鲜度。**
小 + 新 可以击败 大 + 旧。